LRU Cache Replacement Policy Implementation Guide

This guide provides an overview of the key functions that should be implemented for a **Least Recently Used (LRU)** cache replacement policy. The guide explains the purpose and functionality of each function, aiding in the development of the LRU cache replacement policy.

Constructor (LRU::LRU)

- **Purpose**: Initialize the LRU replacement policy.
- **Functionality**:
  - Calls the base class constructor with the provided parameters.
  - Sets up internal data structures to track the last access time of each cache line.
  - Ensures the policy is ready to track the least recently accessed cache lines for eviction.

Invalidate (LRU::invalidate)

- **Purpose**: Handles the invalidation of a cache line.
- **Functionality**:
  - Resets the last access time (tickAccessed) of the cache line's replacement data to zero.
  - This indicates that the cache line is no longer valid and can be reused.
- **Implementation Notes**:
  - Use `std::static_pointer_cast` to cast the generic `ReplacementData` pointer to `LRUReplData` pointer.
  - Set `tickAccessed` to `Tick(0)` to represent an invalid state.

Touch (LRU::touch)

- **Purpose**: Updates the replacement data when a cache line is accessed.
- **Functionality**:
  - Updates the last access time (tickAccessed) to the current simulation tick (`curTick()`).
  - This records the time at which the cache line was last accessed, keeping the LRU order up to date.
- **Implementation Notes**:
  - Use `std::static_pointer_cast` to cast the replacement data pointer to `LRUReplData`.
  - Update `tickAccessed` with `curTick()` to capture the current simulation time.

Reset (LRU::reset)

- **Purpose**: Updates the replacement data when a cache line is inserted or reinserted.

- **Functionality**:
  - Sets the last access time (`tickAccessed`) of the cache line's replacement data to the current simulation tick (`curTick()`).
  - This records the time at which the cache line was inserted or reinserted, ensuring it is correctly tracked in the LRU order.
- **Implementation Notes**:
  - Use `std::static_pointer_cast` to cast the replacement data pointer to `LRUReplData`.
  - Update `tickAccessed` with `curTick()` to record the current simulation time.

Get Victim (LRU::getVictim)

- **Purpose**: Select a cache line to evict based on the LRU policy.
- **Functionality**:
  - Iterates over the list of replacement candidates.
  - Compares the `tickAccessed` values of each candidate.
  - Selects the cache line with the smallest `tickAccessed` value (i.e., the least recently accessed cache line) as the victim.
- **Implementation Notes**:
  - Ensure that there is at least one candidate on the list.
  - Use `assert` to verify that the candidate list is not empty.
  - Use `std::static_pointer_cast` to access the `tickAccessed` of each candidate's replacement data.
  - Return the `ReplaceableEntry` pointer corresponding to the selected victim.

Instantiate Entry (LRU::instantiateEntry)

- **Purpose**: Creates a new replacement data object for a cache line.
- **Functionality**:
  - Allocates and returns a `shared_ptr` to a new `LRUReplData` object.
  - Initializes `tickAccessed` to zero or an appropriate initial value.
- **Implementation Notes**:
  - Use `std::shared_ptr<ReplacementData>` to create and return the new replacement data object.
  - The `LRUReplData` class should contain a member variable, `Tick tickAccessed`, to store the last access time.

Additional Notes

- **Replacement Data Structure (LRUReplData)**:

    - Stores the last access time of a cache line.
    - Used by the LRU policy to determine the eviction order based on recent usage.

- **Simulation Tick (curTick())**:

    - Represents the current simulation time.
    - Used to timestamp cache line accesses accurately.

- **Type Casting**:

    - Use `std::static_pointer_cast` to cast between `ReplacementData` and `LRUReplData`.
    - Necessary for accessing `tickAccessed` within the replacement data.

Example Usage

- **Invalidation**:

  ```
  void LRU::invalidate(const std::shared_ptr<ReplacementData>& replacement_data)
  const {

     std::static_pointer_cast<LRUReplData>(replacement_data)->tickAccessed = Tick(0);

  }
  ```

- **Resetting Last Access Time**:

  ```
  void LRU::reset(const std::shared_ptr<ReplacementData>& replacement_data) const {

     std::static_pointer_cast<LRUReplData>(replacement_data)->tickAccessed =
  curTick();

  }
  ```

- **Selecting Victim**:

  ```
  ReplaceableEntry* LRU::getVictim(const ReplacementCandidates& candidates) const {

     assert(candidates.size() > 0);

     ReplaceableEntry* victim = candidates[0];
  ```

```
    for (const auto& candidate : candidates) {

        if
(std::static_pointer_cast<LRUReplData>(candidate->replacementData)->tickAccessed <


std::static_pointer_cast<LRUReplData>(victim->replacementData)->tickAccessed) {

            victim = candidate;

        }

    }

    return victim;

}
```

Summary

The LRU cache replacement policy evicts the least recently used cache line, ensuring that more frequently accessed lines stay in the cache. The key to implementing this policy is accurately tracking the access times of cache lines and selecting the one with the oldest access time for eviction.